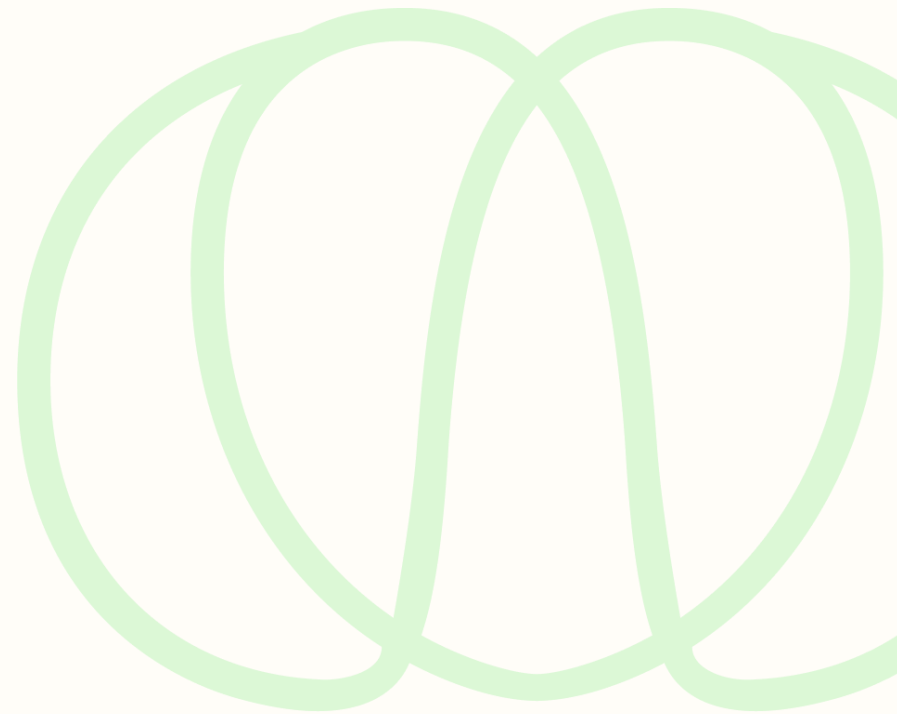


ada

Datas com Java

Pacote `java.time` — Java 8+



Problemas da API legada

Problema	Detalhe
Meses indexados em 0	Janeiro = 0, Dezembro = 11
Ano relativo a 1900	2024 → passar 124
Classes mutáveis	Perigoso em código concorrente
API confusa e verbosa	Difícil de ler e manter
Sem suporte a fusos	Necessitava de gambiarras

Java 8 resolveu tudo isso com o pacote `java.time`

02

O pacote `java.time`



java.time — Princípios do design

A nova API foi inspirada na **Joda-Time** e projetada para ser:

- ▶ **Imutável** — objetos nunca mudam após criados → thread-safe
- ▶ **Fluente** — métodos podem ser encadeados
- ▶ **Clara** — nomes intuitivos, sem ambiguidade

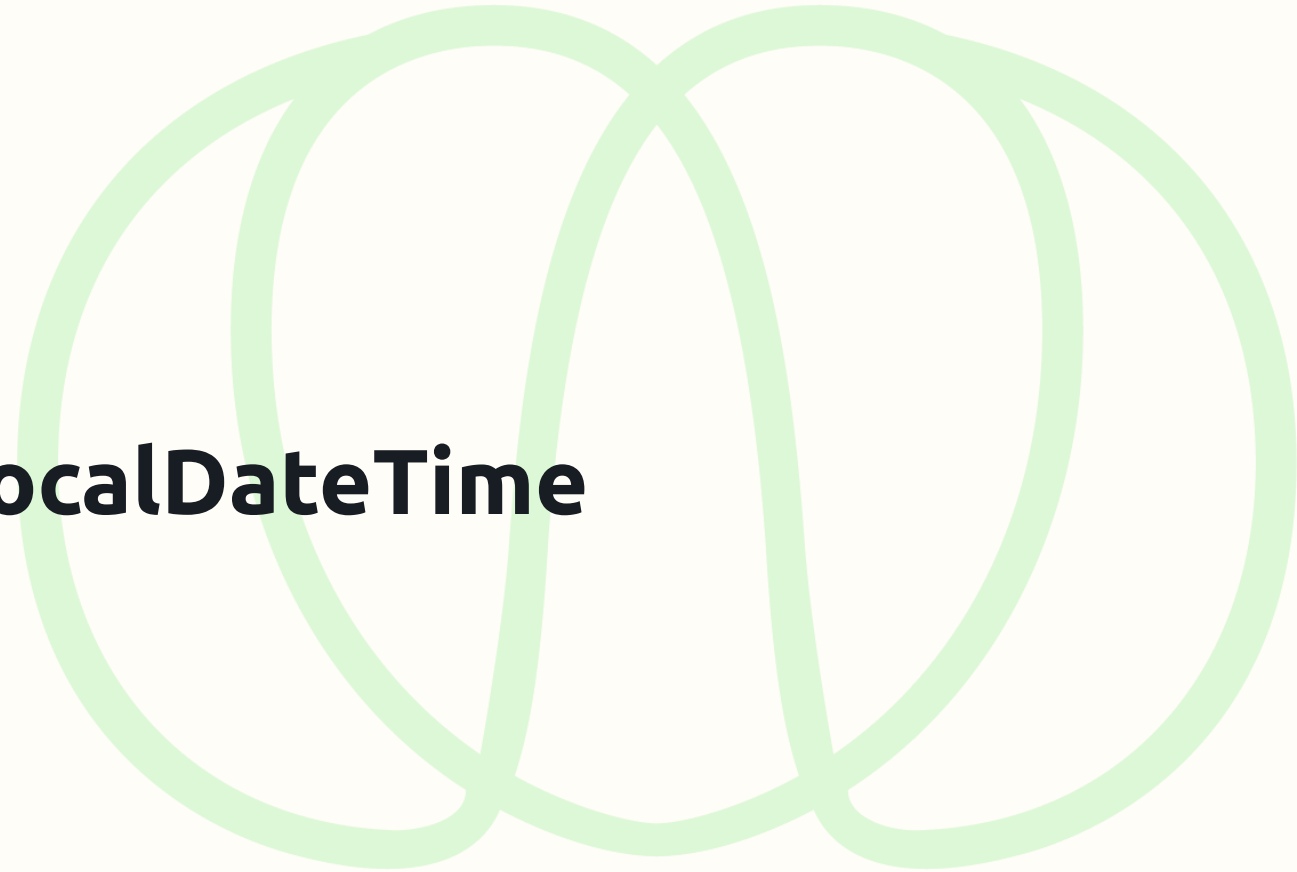
```
// Nova filosofia: cada operação retorna um NOVO objeto
LocalDate hoje          = LocalDate.now();
LocalDate semanaQueVem = hoje.plusWeeks(1); // hoje não muda!
LocalDate proximoMes    = hoje.plusMonths(1); // encadeável
```

Principais classes do java.time

Classe	O que representa
LocalDate	Apenas data (ano, mês, dia)
LocalTime	Apenas hora (hora, minuto, segundo, nano)
LocalDateTime	Data + hora, sem fuso horário
ZonedDateTime	Data + hora + fuso horário
OffsetDateTime	Data + hora + offset UTC fixo
Instant	Ponto no tempo em UTC (timestamp)
Period	Intervalo em anos, meses e dias
Duration	Intervalo em horas, minutos e segundos
DateTimeFormatter	Formatar e parsear

03

LocalDate, LocalTime e LocalDateTime



Capturando o momento atual — `now()`

```
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;

public class InstanciandoDatas {
    public static void main(String[] args) {
        LocalDate data = LocalDate.now();
        LocalTime horario = LocalTime.now();

        LocalDateTime dataHora = LocalDateTime.of(data, horario);
        System.out.println("of(data, horario): " + dataHora);

        dataHora = LocalDateTime.now();
        System.out.println("now(): " + dataHora);
    }
}
```

Data e hora usando `LocalDateTime.of(data, horario)`: 2022-05-04T18:32:49.130
Data e hora usando `LocalDateTime.now()`: 2022-05-04T18:32:49.132

Criando datas específicas — of()

Método	Retorno
<code>LocalDate.of(YYYY, MM, DD)</code>	Data com ano, mês e dia
<code>LocalTime.of(hh, mm, ss)</code>	Hora com horas, minutos e segundos
<code>LocalDateTime.of(YYYY, MM, DD, hh, mm, ss)</code>	Data e hora completas

```
LocalDate aniversario = LocalDate.of(1995, 6, 20);
LocalDate natal       = LocalDate.of(2024, Month.DECEMBER, 25);

LocalTime reuniao     = LocalTime.of(14, 30, 0);
LocalTime preciso     = LocalTime.of(9, 15, 30, 500_000_000);

LocalDateTime evento  = LocalDateTime.of(2024, 12, 25, 20, 0, 0);
```

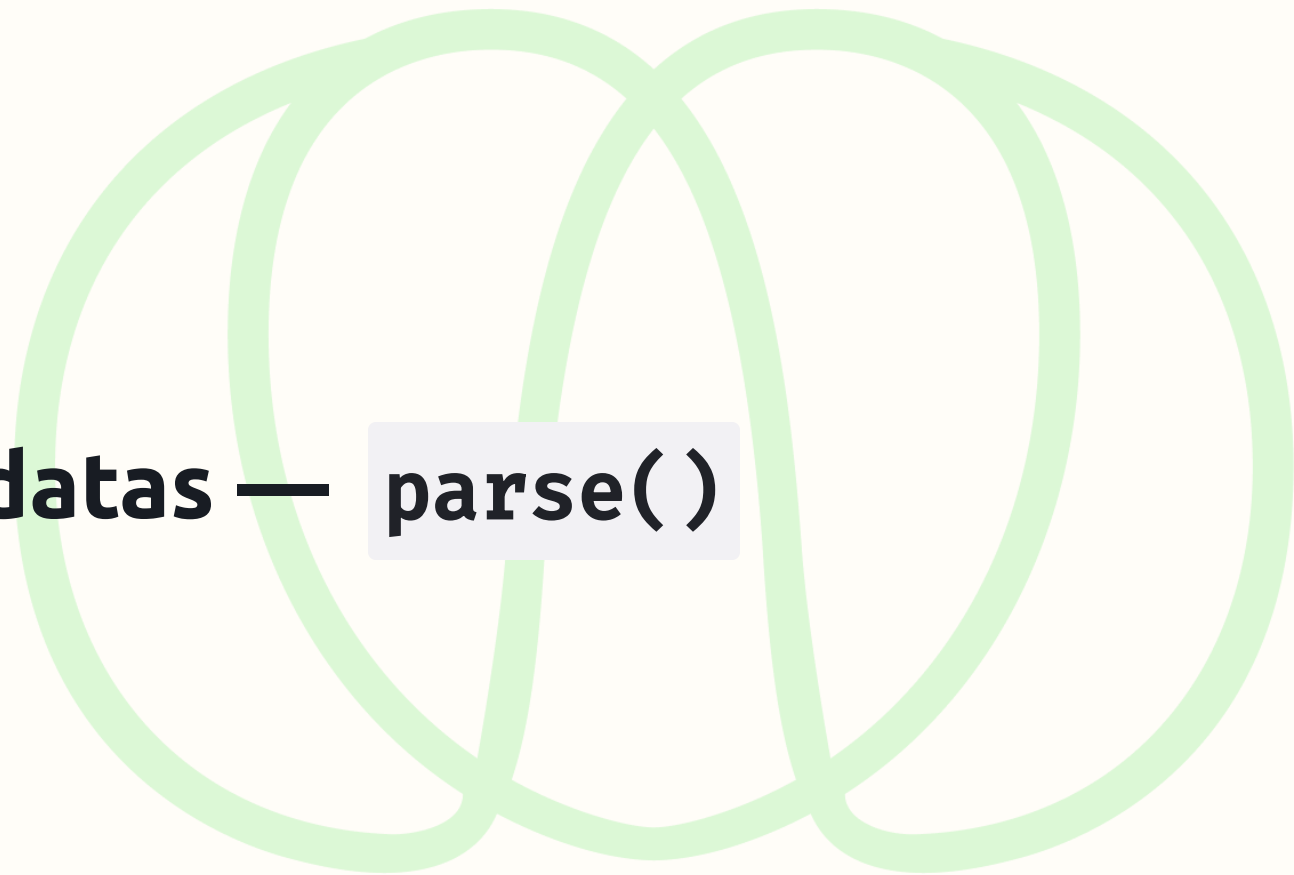
O que podemos extrair de uma data?

```
LocalDate hoje = LocalDate.now(); // 2024-04-24

System.out.println(hoje.getYear());           // 2024
System.out.println(hoje.getMonth());          // APRIL
System.out.println(hoje.getMonthValue());     // 4
System.out.println(hoje.getDayOfMonth());     // 24
System.out.println(hoje.getDayOfWeek());      // WEDNESDAY
System.out.println(hoje.getDayOfYear());      // 115
System.out.println(hoje.isLeapYear());        // true
```

04

Convertendo Strings em datas — `parse()`



Método `parse()` — formato ISO 8601

Por padrão, o `parse()` espera o formato **ISO 8601**:

Método	Formato esperado	Exemplo
<code>LocalDate.parse(...)</code>	<code>"YYYY-MM-DD"</code>	<code>"2022-01-30"</code>
<code>LocalTime.parse(...)</code>	<code>"hh:mm:ss"</code>	<code>"09:47:55"</code>
<code>LocalDateTime.parse(...)</code>	<code>"YYYY-MM-DDThh:mm:ss"</code>	<code>"2022-12-20T05:45:43"</code>

Note o **T** separando data e hora no `LocalDateTime`

Exemplo completo de parse()

```
public class DatasCustomizadas {  
    public static void main(String[] args) {  
        LocalDate dataInicio = LocalDate.of(2022, 1, 1);  
        LocalDate dataFim    = LocalDate.parse("2022-01-30");  
        System.out.println("Ferias de " + dataInicio + " ate " + dataFim);  
  
        LocalTime horaInicio = LocalTime.of(8, 15, 30);  
        LocalTime horaFim    = LocalTime.parse("09:47:55");  
        System.out.println("Intervalo de " + horaInicio + " ate " + horaFim);  
  
        LocalDateTime inicio = LocalDateTime.of(2022, 12, 15, 1, 22, 43);  
        LocalDateTime fim     = LocalDateTime.parse("2022-12-20T05:45:43");  
        System.out.println("Recesso de " + inicio + " ate " + fim);  
    }  
}
```

Ferias de 2022-01-01 ate 2022-01-30
Intervalo de 08:15:30 ate 09:47:55
Recesso de 2022-12-15T01:22:43 ate 2022-12-20T05:45:43

05

Formatando datas — DateFormatter



DateTimeFormatter — visão geral

Armazena formatos personalizados. Criado com:

```
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
```

Dois usos principais:

Sentido	Método	Descrição
Data → String	<code>.format(formatter)</code>	Formata uma data como texto
String → Data	<code>.parse(texto, formatter)</code>	Converte texto em data

Referência de padrões de formato

Símbolo	Significado	Exemplo
yyyy / yy	Ano 4 ou 2 dígitos	2024 / 24
MM / MMM / MMMM	Mês número / abrev. / extenso	04 / abr / abril
dd / d	Dia com / sem zero	07 / 7
HH / hh	Hora 24h / 12h	14 / 02
mm	Minuto	30
ss	Segundo	05
EEE / EEEE	Dia da semana abrev. / extenso	qui / quinta-feira

Exemplo: `parse()` e `format()`

```
public class FormatandoDatas {  
    public static void main(String[] args) {  
        String strDataBR = "25-06-1980 11:15";  
        DateTimeFormatter formatoBR = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");  
  
        // String → LocalDateTime  
        LocalDateTime dataHora = LocalDateTime.parse(strDataBR, formatoBR);  
        System.out.println("Extraído do formato BR: " + dataHora);  
  
        // LocalDateTime → String em outro formato  
        DateTimeFormatter outroFormato = DateTimeFormatter.ofPattern("dd MM yy - HH, mm");  
        System.out.println("Outro formato: " + dataHora.format(outroFormato));  
    }  
}
```

Data e hora extraídos do formato BR: 1980-06-25T11:15
Outros formatos: 25 06 80 - 11, 15

06

Manipulando datas



Operações com datas

As classes `java.time` são **imutáveis** — todo método de modificação retorna um **novo objeto**.

```
public class ManipulandoDatas {  
    public static void main(String[] args) {  
        LocalDate dataNascimento = LocalDate.of(1980, 6, 25);  
  
        System.out.println("Dia da semana: " + dataNascimento.getDayOfWeek());  
        System.out.println("3 dias antes: " + dataNascimento.minusDays(3));  
        System.out.println("3 dias depois: " + dataNascimento.plusDays(3));  
    }  
}
```

```
Dia da Semana: WEDNESDAY  
Três dias antes: 1980-06-22  
Três dias depois: 1980-06-28
```

Principais métodos de ajuste

Classe	Métodos de ajuste
LocalDate	minusDays(n) · plusWeeks(n) · withYear(n)
LocalTime	minus(n, ChronoUnit.MINUTES) · plusMinutes(n) · withHour(n)
LocalDateTime	minusDays(n) · plusMinutes(n) · plus(Duration) · withMonth(n)
ZonedDateTime	withZoneSameInstant(ZoneId)

Armadilha comum:

```
LocalDate data = LocalDate.now();
data.plusDays(5);           // resultado DESCARTADO!
data = data.plusDays(5);    // correto: capturar o novo valor
```

07

Fusos Horários — ZonedDateTime



Fusos horários — conceito

- ▶ Referência: **Greenwich Mean Time (GMT/UTC)**
- ▶ `ZonedDateTime` representa data e hora **com fuso horário**
- ▶ Precisa de um `ZoneId` para identificar o fuso

```
// Identificadores de fuso (String)
ZoneId saoPaulo = ZoneId.of("America/Sao_Paulo"); // UTC-3
ZoneId acre     = ZoneId.of("Brazil/Acre");        // UTC-5
ZoneId lisboa   = ZoneId.of("Europe/Lisbon");      // UTC+0/+1
ZoneId tokyo    = ZoneId.of("Asia/Tokyo");         // UTC+9
```

Exemplo: ZonedDateTime

```
public class DataComFuso {  
    public static void main(String[] args) {  
        LocalDateTime dateTime = LocalDateTime.now();  
  
        ZonedDateTime sp = ZonedDateTime.of(dateTime, ZoneId.of("America/Sao_Paulo"));  
        ZonedDateTime ac = ZonedDateTime.of(dateTime, ZoneId.of("Brazil/Acre"));  
        ZonedDateTime pt = ZonedDateTime.of(dateTime, ZoneId.of("Europe/Lisbon"));  
  
        System.out.println("SP: " + sp);  
        System.out.println("AC: " + ac);  
        System.out.println("PT: " + pt);  
    }  
}
```

```
SP: 2022-05-05T00:34:41.243-03:00[America/Sao_Paulo]  
AC: 2022-05-05T00:34:41.243-05:00[Brazil/Acre]  
PT: 2022-05-05T00:34:41.243+01:00[Europe/Lisbon]
```

Horário de verão — isDaylightSavings()

```
public class HorarioDeVerao {  
    public static void main(String[] args) {  
        ZoneId zoneId = ZoneId.of("America/Sao_Paulo");  
  
        LocalDateTime maio = LocalDateTime.parse("2018-05-01T05:00:00");  
        ZonedDateTime zMaio = ZonedDateTime.of(maio, zoneId);  
        boolean dst = zoneId.getRules().isDaylightSavings(zMaio.toInstant());  
        System.out.println(zMaio + " - horário de verão: " + dst);  
  
        LocalDateTime dez = LocalDateTime.parse("2018-12-01T05:00:00");  
        ZonedDateTime zDez = ZonedDateTime.of(dez, zoneId);  
        dst = zoneId.getRules().isDaylightSavings(zDez.toInstant());  
        System.out.println(zDez + " - horário de verão: " + dst);  
    }  
}
```

2018-05-01T05:00-03:00[America/Sao_Paulo] - horário de verão: **false**
2018-12-01T05:00-02:00[America/Sao_Paulo] - horário de verão: **true**

08

TemporalAdjusters



Ajustes com TemporalAdjusters

Métodos para **ajustes complexos** de datas:

Método	O que faz
<code>firstDayOfMonth()</code>	Primeiro dia do mês atual
<code>lastDayOfMonth()</code>	Último dia do mês atual
<code>firstDayOfNextYear()</code>	Primeiro dia do próximo ano
<code>next(DayOfWeek)</code>	Próxima ocorrência de um dia da semana
<code>previous(DayOfWeek)</code>	Ocorrência anterior de um dia da semana

Exemplo: próxima sexta-feira

```
public class AjustandoDatas {  
    public static void main(String[] args) {  
        LocalDateTime dateTime = LocalDateTime.parse("2018-05-01T05:00:00");  
        ZonedDateTime zdt = ZonedDateTime.of(dateTime, ZoneId.of("America/Sao_Paulo"));  
  
        System.out.println("Data: " + zdt);  
        System.out.println("Dia: " + zdt.getDayOfWeek());  
  
        ZonedDateTime proximaSexta = zdt.with(TemporalAdjusters.next(DayOfWeek.FRIDAY));  
        System.out.println("Próxima sexta: " + proximaSexta);  
    }  
}
```

```
Data: 2018-05-01T05:00-03:00[America/Sao_Paulo]  
      Dia: TUESDAY  
Próxima sexta: 2018-05-04T05:00-03:00[America/Sao_Paulo]
```

09

Period, Duration e Instant



Period — diferença em dias, meses e anos

```
public class UsandoPeriodosDeTempo {  
    public static void main(String[] args) {  
        ZonedDateTime inicio = ZonedDateTime.of(  
            2022, 4, 8, 13, 35, 56, 0, ZoneId.of("America/Sao_Paulo")  
        );  
  
        Period tresMeses = Period.ofMonths(3);  
        ZonedDateTime fim = inicio.plus(tresMeses);  
  
        System.out.println("Início: " + inicio);  
        System.out.println("Fim (+ 3 meses): " + fim);  
        System.out.println("Fim em Lisboa: " +  
            fim.withZoneSameInstant(ZoneId.of("Europe/Lisbon")));  
    }  
}
```

```
Inicio: 2022-04-08T13:35:56-03:00[America/Sao_Paulo]  
Fim (+ 3 meses): 2022-07-08T13:35:56-03:00[America/Sao_Paulo]  
Fim em Lisboa: 2022-07-08T17:35:56+01:00[Europe/Lisbon]
```

Duration — diferença em horas e minutos

```
public class DuracaoDeIntervalo {  
    public static void main(String[] args) {  
        LocalDateTime begins = LocalDateTime.of(12, 7, 10);  
        LocalDateTime ends   = LocalDateTime.of(14, 22, 37);  
        System.out.println("Intervalo: " + begins + " - " + ends);  
  
        long minutes = ChronoUnit.MINUTES.between(begins, ends);  
        System.out.println("Total minutos: " + minutes);  
  
        Duration duration = Duration.ofMinutes(minutes);  
        System.out.println("Duração: " + duration);  
    }  
}
```

Intervalo: 12:07:10 - 14:22:37
Total minutos: 135
Duração: PT2H15M

Instant — timestamp desde a Epoch

Instant representa um ponto no tempo como **segundos desde 01/01/1970** (Unix Epoch).

```
public class UsandoInstant {  
    public static void main(String[] args) {  
        ZonedDateTime evento = ZonedDateTime.of(  
            1998, 1, 13, 16, 45, 56, 0, ZoneId.of("America/Sao_Paulo")  
        );  
        Instant eventInstant = evento.toInstant();  
  
        System.out.println("Data/hora: " + evento);  
        System.out.println("Instante: " + eventInstant);  
        System.out.println("Segundos desde 01/01/1970: " + eventInstant.getEpochSecond());  
    }  
}
```

```
Data/hora: 1998-01-13T16:45:56-02:00[America/Sao_Paulo]  
Instante: 1998-01-13T18:45:56Z  
Segundos desde 01/01/1970: 884717156
```

Quando usar cada um?

Situação	Classe
Diferença entre datas (meses de contrato, idade)	Period
Duração de tempo curto (reunião, intervalo)	Duration
Registrar quando algo aconteceu (log, auditoria)	Instant
Medir tempo de execução de código	Instant

10

Internacionalização com Locale



Locale — formatos regionais

`java.util.Locale` representa uma região cultural. Pode ser usada com `DateTimeFormatter` para adaptar a saída ao idioma local.

```
Locale locBR = new Locale("pt", "BR"); // Português - Brasil
Locale locJA = new Locale("ja");      // Japonês
Locale locUS = new Locale("US");      // Inglês - EUA

// Construtores disponíveis
new Locale(String language)
new Locale(String language, String country)
new Locale(String language, String country, String variant)
```

Exemplo: Locale com DateTimeFormatter

```
public class UsandoLocale {  
    public static void main(String[] args) {  
        ZonedDateTime zdt = ZonedDateTime.now();  
        Locale locBR = new Locale("pt", "BR");  
        Locale locJA = new Locale("ja");  
  
        System.out.println("Formato LONG – Brasil: " +  
            zdt.format(DateTimeFormatter  
                .ofLocalizedDateTime(FormatStyle.LONG).withLocale(locBR)));  
  
        System.out.println("Formato LONG – Japão: " +  
            zdt.format(DateTimeFormatter  
                .ofLocalizedDateTime(FormatStyle.LONG).withLocale(locJA)));  
    }  
}
```

Formato LONG – Brasil: 5 de maio de 2022 13:14:16 CEST
Formato LONG – Japão: 2022年5月5日 13:14:16 CEST

11

Resumo



Principais métodos — Parte I

Classe	Exemplos de uso
LocalDate	<code>LocalDate.now()</code> · <code>LocalDate.of(2022, 8, 22)</code> · <code>LocalDate.parse("2022-08-22")</code>
LocalTime	<code>LocalTime.now()</code> · <code>LocalTime.of(11, 20, 37)</code> · <code>LocalTime.parse("11:20:37")</code>
LocalDateTime	<code>LocalDateTime.now()</code> · <code>LocalDateTime.of(aDate, aTime)</code> · <code>LocalDateTime.parse("2022-05-09T11:20:37")</code>
ZonedDateTime	<code>ZonedDateTime.now()</code> · <code>ZonedDateTime.of(aDateTime, ZoneId.of("America/Sao_Paulo"))</code>

Principais métodos — Parte II

Classe	Exemplos de uso
<code>DateTimeFormatter</code>	<code>DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm") . ofLocalizedDateTime(FormatStyle.SHORT).withLocale(locale)</code>
<code>Instant</code>	<code>Instant.now() · zonedDateTime.toInstant()</code>
<code>Duration</code>	<code>Duration.between(time1, time2) · Duration.ofMinutes(5)</code>
<code>Period</code>	<code>Period.between(date1, date2) · Period.ofDays(3)</code>
<code>Locale</code>	<code>Locale.getDefault() · new Locale("pt", "BR")</code>

Ajustes de datas

Classe	Exemplos de ajuste
LocalDate	<code>aDate.minusDays(3) · aDate.plusWeeks(1) · aDate.withYear(2024)</code>
LocalTime	<code>aTime.minus(3, ChronoUnit.MINUTES) · aTime.plusMinutes(3) · aTime.withHour(12)</code>
LocalDateTime	<code>aDateTime.minusDays(3) · aDateTime.plusMinutes(10) · aDateTime.plus(Duration.ofMinutes(5))</code>
ZonedDateTime	<code>zonedDateTime.withZoneSameInstant(ZoneId.of("US/Pacific"))</code>

Mapa do pacote java.time

```
java.time
├── LocalDate      → data sem hora, sem fuso
├── LocalTime      → hora sem data, sem fuso
├── LocalDateTime  → data + hora, sem fuso
├── ZonedDateTime → data + hora + fuso horário
├── OffsetDateTime → data + hora + offset UTC fixo
├── Instant        → timestamp UTC (segundos desde 1970)
├── format/
│   └── DateTimeFormatter → formatar e parsear
├── temporal/
│   └── TemporalAdjusters → ajustes complexos de data
├── Period         → diferença em anos / meses / dias
└── Duration       → diferença em horas / minutos / segundos
```

Referências

- ▶ Pacote java.time — Java 17
- ▶ Classe DateTimeFormatter
- ▶ Classe ZonedDateTime
- ▶ Classe TemporalAdjusters
- ▶ Todas as Zone IDs disponíveis
- ▶ Introduction to the Java 8 Date/Time API — Baeldung
- ▶ SIERRA, K.; BATES, B.; ROBSON, E. **OCJP Java SE 8 Programmer II Exam Guide**, Cap. 4, 2018.

```
print("thank you")
```

Módulo BE-JV-005 • Técnicas de Programação I • Ada Tech